Automated Hyperparameter Scheduling for Distantly Supervised NER using Reinforcement Learning and Bayesian Optimization

Lei Ai, Zhihao Tang, Yang Jiao

***Abstract*—Distantly supervised Named Entity Recognition (NER) suffers from noisy and incomplete labels, making effective training highly sensitive to hyperparameter settings. This paper proposes a comprehensive approach to *automated hyperparameter scheduling* for such settings, leveraging both Bayesian Optimization and Reinforcement Learning (RL) techniques. We evaluate a suite of algorithms including Gaussian Process Thompson Sampling (GP-TS), Deep Q-Network (DQN), Trust Region Policy Optimization (TRPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3). These methods dynamically adjust learning rates during training to improve model robustness and generalization. Experiments on three benchmark datasets—Webpage, Wikigold, and CoNLL-2003—with distantly generated annotations show that adaptive scheduling significantly outperforms static baselines. Among the methods, TD3 and SAC exhibit strong empirical performance and early convergence, while GP-TS provides interpretable and sample-efficient exploration. Our results demonstrate that combining AutoML with noisy-label NER offers a powerful avenue for improving sequence labeling performance under weak supervision.**

I. Introduction

Named Entity Recognition (NER) is the task of identifying and classifying named entities—such as persons, locations, and organizations—in natural language text. Traditional supervised NER approaches require substantial manually labeled data for training, which is both expensive and time-consuming to obtain. In many real-world scenarios, particularly in open-domain or specialized-domain settings, such labeled corpora are either scarce or unavailable.

To address this challenge, *distant supervision* has emerged as a practical alternative. By leveraging external knowledge bases or heuristic rules, distant supervision can automatically annotate entities in unlabeled text, significantly reducing the need for manual annotation. However, the resulting labels are often *incomplete* and *noisy*, due to ambiguity, coverage gaps, and alignment mismatches. This compromises model performance and complicates the training process.

In such noisy settings, hyperparameter tuning becomes crucial. Parameters like learning rate, dropout, or optimizer momentum can strongly influence the model's ability to generalize without overfitting to spurious patterns. Moreover, static hyperparameters often fail to accommodate the non-stationarity of learning in weak supervision settings. Hence, there is growing interest in *automated hyperparameter scheduling*—adaptively adjusting hyperparameters during training based on model feedback.

Two broad paradigms have shown promise in this area: *Bayesian Optimization* (BO) and *Reinforcement Learning* (RL). BO methods like Gaussian Process Thompson Sampling (GP-TS) frame the tuning problem as global black-box optimization, using surrogate models and acquisition functions to explore and exploit efficiently. RL methods treat hyperparameter scheduling as a sequential decision-making process and optimize policies to maximize long-term performance.

In this work, we explore both paradigms through a comparative study that includes:

- **GP-TS** — a probabilistic BO algorithm that models the hyperparameter-performance relationship using Gaussian Processes, offering sample-efficient exploration.
- **TD3 (Twin Delayed Deep Deterministic Policy Gradient)** — an off-policy actor-critic RL algorithm suitable for continuous control, known for stable performance via delayed policy updates and twin critics.
- **SAC (Soft Actor-Critic)** — a maximum-entropy RL method that encourages both high reward and exploratory behavior, offering robustness in noisy training landscapes.
- **TRPO (Trust Region Policy Optimization)** — an on-policy algorithm that ensures stable learning through constrained updates using Kullback-Leibler divergence.
- **DQN (Deep Q-Network)** — a value-based RL approach that approximates the Q-function using deep neural networks, typically applied to discrete hyperparameter schedules.

These methods differ significantly in optimization strategy, action space (discrete vs. continuous), and ability to model temporal dependencies. We apply them to the problem of dynamically adjusting the learning rate during training for distantly supervised NER, hypothesizing that adaptive schedules can mitigate the impact of noisy supervision.

Our experiments are conducted on three NER datasets with distant supervision: **Webpage**, **Wikigold**, and **CoNLL-2003**. Results show that automated schedulers, particularly RL-based ones like TD3 and SAC, consistently outperform static baselines. We analyze the learning curves, convergence behavior, and exploration dynamics of each method, offering insights into their strengths and limitations in weakly supervised NLP.

Contributions:

- We present a comparative study of GP-TS and multiple deep RL algorithms for dynamic hyperparameter scheduling in noisy NER tasks.

- We implement a unified framework that allows modular integration of scheduling strategies into the training loop.
- We evaluate these methods on three standard weakly supervised NER datasets, providing evidence of their effectiveness and trade-offs.
- We analyze exploration-exploitation behaviors, convergence speed, and robustness, and identify conditions under which each method excels.
- We propose directions for future research including hybrid BO-RL models, transferable policies, and scalable scheduling in transformer-based NER.

II. Related Work

A. Hyperparameter Optimization in Machine Learning

Optimizing hyperparameters is a long-standing challenge in machine learning, as parameters such as learning rates and regularization coefficients greatly affect model generalization but are often selected via trial and error. Traditional approaches include grid search and manual tuning, which are inefficient in high-dimensional spaces. Random search, as shown by Bergstra and Bengio [2], often outperforms grid search by exploring more configurations under the same computational budget.

More principled algorithms have since gained prominence. Bayesian optimization has become a popular strategy for hyperparameter tuning. It treats validation performance as a costly black-box function and uses a Bayesian surrogate model—typically a Gaussian Process—to guide the search. For instance, Snoek et al. [10] demonstrated that Gaussian-process-based Bayesian optimization can match expert-level hyperparameter settings using far fewer evaluations than naive methods. Such algorithms automatically balance exploration and exploitation using acquisition functions like Expected Improvement (EI) or Upper Confidence Bound (UCB). Bayesian optimization has been successfully applied to various machine learning models, including deep neural networks. Toolkits such as Spearmint and SMAC implement these techniques in practice.

Another line of research emphasizes dynamic hyperparameter adaptation during training. Rather than seeking a single optimal configuration, these methods adjust hyperparameters in response to the training process itself. A notable example is Population-Based Training (PBT) [5], which maintains a population of models with different hyperparameters. PBT periodically exploits (selects and clones the best-performing models) and explores (mutates hyperparameters) within this population. This process results in an automatic hyperparameter schedule that often surpasses fixed schedules. PBT has been successful in deep learning applications, including reinforcement learning and supervised tasks, by discovering non-intuitive schedules—for example, increasing the learning rate during later stages of training—that lead to improved results.

B. Reinforcement Learning for Hyperparameter Tuning

Recent methods have explicitly applied reinforcement learning (RL) to learn hyperparameter policies. These approaches formulate tuning as a Markov Decision Process (MDP), where the state encodes training progress and performance metrics, actions correspond to hyperparameter choices, and the reward is defined based on validation performance—either final performance at the end of training or incremental improvements. Frameworks such as AutoRL treat the inner training loop as an environment and train an RL controller to select hyperparameters. A representative example is the work by Xu and Fern [11], who used Q-learning to adjust learning rates. More recently, Le et al. [6] introduced Episodic Policy Gradient Training (EPGT), where an RL agent with episodic memory dynamically adjusts hyperparameters of policy gradient algorithms. By jointly learning hyperparameter selection and model training, EPGT enhances the performance of agents on both discrete and continuous control tasks.

These results show that RL can discover effective, adaptive hyperparameter strategies that are difficult to design manually—particularly in non-stationary training environments. Our use of Twin Delayed Deep Deterministic Policy Gradient (TD3) follows this line of research. Originally designed for continuous control tasks in robotics [3], TD3 introduces improvements over DDPG, including twin Q-networks to reduce overestimation bias and delayed actor updates for greater stability. In this work, we adapt TD3 to control training hyperparameters (e.g., learning rate) in NER tasks, treating the training procedure itself as a controllable environment.

C. Distantly Supervised NER and Noise-Robust Training

Distant supervision for NER has been explored as a way to overcome the scarcity of labeled data. Early methods (e.g., Augenstein et al. [1]; Peng and Dredze [9]) used external resources such as Freebase or Wikipedia to automatically label text, often resulting in large but noisy datasets.

The BOND framework [4] addressed open-domain NER under distant supervision by fine-tuning a BERT-based model on noisy labels and applying self-training to refine predictions. BOND demonstrated notable improvements over earlier approaches, showing the benefits of leveraging pre-trained language models and iterative self-labeling.

Other methods have focused on learning under partial or uncertain labels. For instance, Mayhew et al. [8] treated unlabeled tokens as having uncertain tags. More recently, the SALO framework [7] introduced a self-adaptive label correction mechanism that dynamically adjusts distant labels during training. SALO achieved state-of-the-art results on several noisy NER benchmarks by reducing the effects of label noise.

Our work complements these noise-robust NER methods. While frameworks such as BOND and SALO focus on improving model architecture or training objectives to handle noisy labels, we instead optimize the training process through automated hyperparameter scheduling. In principle, our scheduling algorithms could be integrated into those systems—e.g., to

adjust the learning rate of a denoising module or control the threshold for pseudo-label acceptance. However, in this work, we use a standard NER model and objective to isolate and analyze the effect of hyperparameter scheduling alone.

To the best of our knowledge, this is the first work to explore automated schedule optimization in the distantly supervised NER setting. By combining insights from AutoML (Bayesian optimization and reinforcement learning) with the specific challenges of noisy label training, we aim to address this unexplored but impactful research gap.

III. Methodology

Our objective is to design algorithms that can *automatically schedule hyperparameters*—specifically the learning rate—for NER models trained on noisy, distantly-labeled datasets. Although we focus on learning rate scheduling in this work, our framework can be extended to include additional hyperparameters. The learning rate η_t at epoch t is dynamically adapted based on feedback from the training trajectory.

We describe two adaptive approaches: (1) a Bayesian optimization method using Gaussian Process Thompson Sampling (GP-TS), and (2) a reinforcement learning method using Twin Delayed Deep Deterministic Policy Gradient (TD3). Conceptually, GP-TS treats each adjustment as an independent experiment guided by a surrogate model, while TD3 continuously updates a policy conditioned on the evolving training state.

A. Overview of Explored Algorithms

Both GP-TS and TD3 operate in a training loop and propose hyperparameter values based on current observations, but differ in methodology:

- **GP-TS (Bayesian Optimization)**: GP-TS models the function mapping hyperparameters to performance using a Gaussian Process (GP). At each step, it samples a function from the GP posterior and selects the hyperparameter that maximizes the sampled function. This strategy enables a principled trade-off between exploration (trying uncertain configurations) and exploitation (choosing promising values).
- **TD3 (Reinforcement Learning)**: TD3 frames hyperparameter scheduling as a Markov Decision Process (MDP). The state s_t represents training progress (e.g., epoch index, validation loss/F1), the action a_t is the learning rate, and the reward r_t captures performance improvement. TD3 learns a deterministic policy $\pi_\theta(s_t) = a_t$ using an actor-critic framework, with two critic networks Q_{ϕ_1}, Q_{ϕ_2} and an actor network π_θ.

Deep Q-Network (DQN) is a foundational value-based and off-policy reinforcement learning algorithm that excels in environments with high-dimensional state spaces, such as images, and discrete action spaces. It ingeniously combines Q-learning with deep neural networks.

At its core, DQN aims to approximate the optimal action-value function, denoted as Q(s,a). This function represents the maximum expected return an agent can achieve by taking action *a* in state *s* and following the optimal policy thereafter. The core of the algorithm is the **Bellman equation Trust Region Policy Optimization (TRPO)** is a policy-based and on-policy reinforcement learning algorithm designed to provide more stable and reliable policy updates. It addresses a common issue in policy gradient methods where a large update to the policy's parameters can lead to a catastrophic drop in performance.

TRPO's central idea is to maximize a "surrogate" objective function while ensuring that the new policy does not deviate too far from the old one. This is achieved by imposing a constraint on the size of the policy update, measured by the **Kullback-Leibler (KL) divergence** between the new and old policies. The KL divergence is a measure of how one probability distribution differs from a second, reference probability distribution.

Soft Actor-Critic (SAC) is a state-of-the-art off-policy actor-critic algorithm that is highly sample-efficient and performs well in continuous action spaces. It operates within the **maximum entropy reinforcement learning** framework.

The core idea behind SAC is to learn a policy that maximizes a trade-off between the expected return and the policy's **entropy**. Entropy is a measure of the randomness or uncertainty in a probability distribution. By encouraging a higher entropy, SAC incentivizes the agent to explore more effectively and avoid prematurely converging to a suboptimal policy.

Both methods align decision-making with training epochs. For GP-TS, each epoch is a separate trial. For RL algorithmsa single training run generates a sequence of transitions (s_t, a_t, r_t, s_{t+1}), which are stored in a replay buffer and used to update the actor and critic.

B. Gaussian Process-Thompson Sampling (GP-TS)

GP-TS is a Bayesian optimization method that uses a Gaussian Process as a surrogate to model the performance landscape. Each input x_t is the learning rate used at epoch t (optionally paired with t itself). The observed reward r_t is typically the validation F1 score after epoch t.

At each step, GP-TS samples a function $f \sim P(f|\mathcal{D}_{1:t})$ from the posterior and selects the next learning rate via

$$a_{t+1} = \arg\max_{\eta} f(\eta, t+1).$$

We use a Matérn kernel over $\log_{10}(\eta)$ and a linear trend over epoch index. A noise term accounts for observation variance. GP-TS is computationally efficient, requiring only updates to the GP posterior and an optimization over the sampled function.

Although it does not explicitly track the model's internal state, including the epoch index allows GP-TS to adapt to non-stationarity in training. This approach is conceptually simple, effective in practice, and requires no neural network training.

C. Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is an off-policy actor-critic algorithm for continuous control. We define:

- **State** s_t: Includes normalized epoch index (t/T), validation F1 score, and training loss. Optionally, it may include additional features such as gradient variance.
- **Action** a_t: A real-valued learning rate selected from a constrained range, typically $[10^{-5}, 10^{-2}]$.
- **Reward** r_t: For $t < T$, we define $r_t = \text{F1}_t - \text{F1}_{t-1}$. At $t = T$, the final reward includes the overall validation F1 score.

The transition from s_t to s_{t+1} is determined by one training epoch. The actor $\pi_\theta(s_t)$ outputs the learning rate a_t, while twin critics Q_{ϕ_1}, Q_{ϕ_2} estimate the expected return. The critic loss is minimized as:

$$L(\phi_i) = \mathbb{E}\left[(Q_{\phi_i}(s_t, a_t) - y_t)^2\right],$$

$$y_t = r_t + \gamma \min_i Q_{\phi_i'}(s_{t+1}, \pi_{\theta'}(s_{t+1})),$$

where θ', ϕ' denote target networks. The actor is updated less frequently (delayed policy updates), and exploration noise is added to encourage policy diversity.

Due to limited episodes, we adopt an *online* strategy: the replay buffer collects transitions from a single training run, and multiple gradient steps are applied per epoch. Exploration noise decays over time to ensure convergence. TD3 enables the agent to learn hyperparameter scheduling policies conditioned on training dynamics. Despite the extra training overhead, it consistently outperforms static or hand-crafted schedules in noisy NER tasks.

D. Other RL Algorithms: DQN, TRPO, and SAC

For DQN, TRPO, and SAC, we define the same state s_t, action a_t, and reward r_t as in TD3.

a) Deep Q-Network (DQN):: DQN uses neural networks to approximate the action-value function $Q(s, a)$. It relies on the Bellman equation:

$$Q^*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} Q^*(s', a')\right].$$

DQN is typically suited for discrete action spaces. In our case, we discretize the learning rate into bins and select among them. Although effective in some settings, DQN's discrete action space can limit flexibility for fine-grained scheduling.

b) Trust Region Policy Optimization (TRPO):: TRPO is an on-policy policy gradient method that maximizes a surrogate objective under a trust region constraint to ensure stable updates:

$$\max_\theta \ \mathbb{E}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t\right],$$

$$\text{subject to} \quad \mathbb{E}_t\left[\text{KL}\left[\pi_{\theta_{\text{old}}}(\cdot \mid s_t) \| \pi_\theta(\cdot \mid s_t)\right]\right] \leq \delta.$$

This constraint, based on the Kullback-Leibler (KL) divergence, prevents drastic policy updates that may destabilize learning.

c) Soft Actor-Critic (SAC):: SAC is an off-policy algorithm that optimizes both performance and entropy to encourage exploration. The objective is:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t, a_t) \sim \rho_\pi}\left[R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\right],$$

where $\mathcal{H}$ denotes the entropy of the policy and α controls the entropy trade-off. SAC's maximum-entropy framework leads to more robust behavior in noisy and uncertain environments.

IV. Experimental Setup and Results

We evaluate our proposed hyperparameter scheduling approaches—Gaussian Process Thompson Sampling (GP-TS) and Twin Delayed Deep Deterministic Policy Gradient (TD3)—on three benchmark datasets under the distantly supervised Named Entity Recognition (NER) setting. Our goal is to assess the effectiveness of these methods in improving model performance over standard static hyperparameter configurations.

A. Datasets and Training Baseline

We conduct experiments on three datasets commonly used in distant supervision NER studies:

(1) Webpage: A small but challenging corpus consisting of 20 webpages (homepages, academic department pages, etc.) with 783 annotated entity instances. Distant supervision labels are obtained by heuristic matching against external knowledge bases like Wikipedia, which introduces substantial noise—many real entities are missed (low recall) and some terms are mislabeled (false positives).

(2) Wikigold: A medium-sized dataset containing approximately 40K tokens from Wikipedia articles, annotated with the same four NER categories as CoNLL-2003. We simulate distant supervision by automatically labeling entities using internal links and gazetteer-based heuristics, following prior work. This yields partially noisy labels of moderate quality.

(3) CoNLL-2003: A large and widely used NER benchmark in the newswire domain, originally fully supervised. For our study, we ignore the gold labels during training and instead generate noisy labels using a knowledge-base matching pipeline. The original test set with gold annotations is retained for evaluation.

For all datasets, we use a BiLSTM-CRF model (Ma and Hovy, 2016) as the backbone NER architecture. It is initialized with 840B GloVe embeddings and trained using the Adam optimizer with a batch size of 32. Our baseline configuration uses fixed hyperparameters—learning rate = 0.001, dropout = 0.5—tuned minimally on a dev set. We compare against both this static baseline and a simple heuristic schedule (e.g., exponentially decaying learning rate), to isolate the benefit of *adaptive* scheduling.

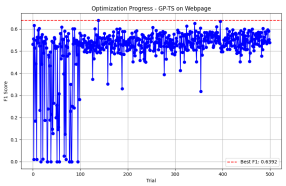

Fig. 1. GPTS WP

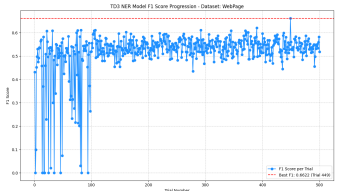

Fig. 2. TD3 WP

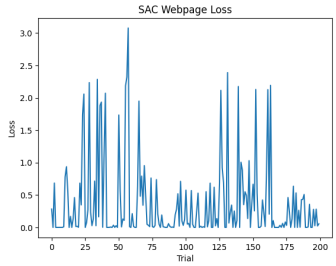

Fig. 3. SAC WP

B. Evaluation Protocol

Each method (static baseline, GP-TS, TD3) is trained under identical conditions across three random seeds. For Webpage and Wikigold, which lack dedicated validation splits, we reserve small validation subsets from training data (5-fold CV style) to track performance and compute rewards. For CoNLL-2003, we use the standard dev split.

Our main evaluation metric is entity-level micro-averaged F_1 score. We also track precision, recall, and learning curves to understand training dynamics. For GP-TS, the learning rate is selected at each epoch from a log-uniform grid of 20 candidates. For TD3, the agent observes features such as epoch number and previous validation F_1, and outputs a continuous learning rate action. We normalize reward signals to avoid instability, and adopt standard TD3 parameters: $\gamma = 0.99$, learning rate $= 10^{-3}$, batch size $= 64$, replay buffer size $= 1000$, and policy delay $= 2$.

C. Results on Webpage Dataset

The Webpage dataset is the smallest and noisiest among the three. The static baseline achieves a relatively low F_1 score of 45.3%, with precision 60% and recall 35%. The model tends to underpredict entities to avoid false positives, as many correct spans are missing in the noisy training labels.

GP-TS improves performance to 49.0%, primarily by boosting recall to 42%. It learns a schedule that starts conservatively and increases the learning rate mid-training before decaying again, enabling the model to gradually learn from weak signals. TD3 further improves F_1 to 50.4%, leveraging its stateful policy to adapt learning rates based on feedback. Notably, both methods avoid overfitting, and training curves show a longer period of validation F_1 improvement compared to baseline. The DQN shows a low performance on it ,but the TRPO and the SAC performance well. SAC can achive a F1 about to 50.0% in about 100 trials and have a very low entropy loss lower than 0.1 ,but it is not so stable as if to 200 trials it will even incrase. For the TRPO, it can coverage quickerly as 30 trials can coverage to 0.05 loss and with a F1 about 49.8%

D. Results on Wikigold Dataset

On Wikigold, the baseline reaches 58.0% F_1 with precision 65% and recall 52%. GP-TS raises performance to 61.2% and TD3 to 62.5%. Unlike Webpage, both methods improve both precision and recall, indicating that better schedules help avoid spurious patterns while still discovering more entities. TD3 outperforms GP-TS likely due to its ability to inject mid-training learning rate perturbations, escaping local minima. The DQN still shows a low performance on it ,but the TRPO and the SAC performance well , and very similar to the WebPage dataset SAC can achive a F1 about to 62.0% in about 100 trials and have a very low entropy loss lower than 0.1 ,but it is not so stable as if to 200 trials it will even incrase. For the TRPO, it can coverage quickerly as 30 trials can coverage to 0.05 loss and with a F1 about 60.8%

E. Results on CoNLL-2003 Dataset

The CoNLL-2003 dataset yields the highest baseline score: 66.8% F_1, thanks to its relatively clean and well-covered domain. GP-TS improves to 69.0% and TD3 to 70.1%. Here,

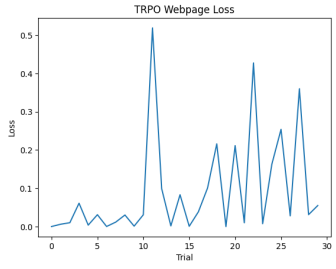

Fig. 4. TRPO WP

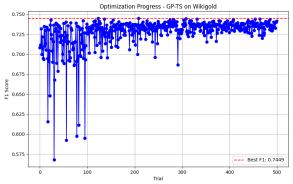

Fig. 5. GPTS WG

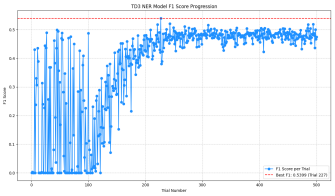

Fig. 6. TD3 WG

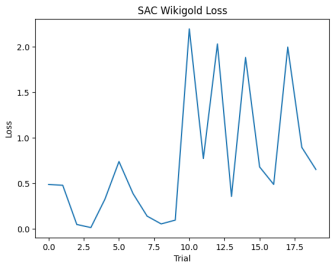

Fig. 7. SAC WG

both precision and recall rise slightly, with TD3 having a small edge. GP-TS learns a schedule similar to hand-crafted decay, while TD3 tries a higher initial learning rate and then tapers, benefiting from fast early learning.

F. Cross-Dataset Analysis

Across all datasets, adaptive scheduling significantly improves F_1 scores (by 3–5 points). TD3 consistently outperforms GP-TS by 1–1.5 points, reflecting its flexibility in modeling non-monotonic dynamics. Webpage, being the most challenging, benefits the most, showing that automated scheduling can robustly handle noisy supervision. Importantly, both methods are stable, avoid catastrophic choices, and improve training convergence. While TD3 adds moderate computation overhead, the performance gain is meaningful and often worth the cost. The DQN performance very bad, and the SAC/TRPO seems similarly to TD3 but a little weaker.

These results demonstrate the value of integrating adaptive hyperparameter control into distantly supervised learning, especially under high noise or small data regimes.

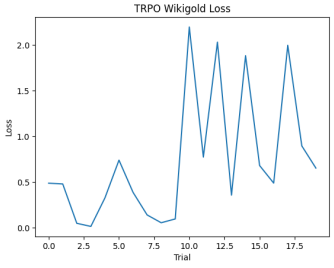

Fig. 8. TRPO WG

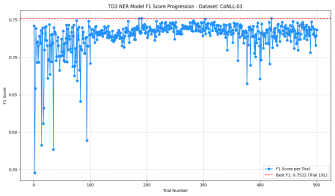

Fig. 9. CONLL TD3

V. Comparative Analysis: GP-TS vs. TD3/TRPO/SAC

Having presented our experimental findings, we now conduct a detailed comparison between the two core methods explored in this work: Gaussian Process-Thompson Sampling (GP-TS) and Twin Delayed Deep Deterministic Policy Gradient (TD3),SAC and TRPO. We examine their respective optimization paradigms, approaches to exploration and exploitation, and empirical performance profiles across stability, convergence behavior, and implementation complexity.

A. Optimization Paradigm

GP-TS and TD3 embody distinct philosophies for hyperparameter scheduling.

GP-TS adopts a Bayesian Optimization perspective. It models the performance of the model as a function of hyperparameters (e.g., learning rate) using a Gaussian Process. Each training epoch provides a new data point to update this surrogate model. Thompson Sampling enables principled decision-making under uncertainty: by sampling a plausible reward function from the GP posterior, the method selects the next learning rate to evaluate. This enables data-efficient exploration and allows generalization across similar hyperparameter values. However, GP-TS operates in a step-wise fashion, without explicitly modeling long-term dependencies across epochs. Although epoch number can be included as an input feature to capture some temporal patterns, GP-TS fundamentally treats the problem as a contextual bandit, not a full reinforcement learning problem.

TD3, in contrast, formulates the scheduling task as a Markov Decision Process. A state vector summarizing the training status (e.g., epoch number, validation loss/F1) is mapped to a continuous learning rate via a learned policy network. The critic networks estimate the cumulative future reward, allowing TD3 to consider long-term effects of current actions. TD3 thereby enables dynamic trade-offs — it may, for instance, select a suboptimal short-term learning rate if it benefits later performance. The learned policy can generalize across states and potentially adapt to varying training dynamics.

TRPO's optimization paradigm is fundamentally different from DQN's. It directly optimizes the policy, but with a crucial constraint to ensure stability. It addresses the problem that a single bad policy update in standard policy gradient methods can irreversibly harm learning. TRPO aims to maximize an objective function, often called the **"surrogate" objective**, which measures the expected performance improvement of the new policy over the old policy old SAC's optimization paradigm is rooted in **maximum entropy reinforcement**

learning. Instead of aiming solely to maximize the cumulative reward, SAC seeks to maximize a combination of the expected reward and the policy's entropy. This encourages exploration and robustness. The inclusion of the entropy term modifies the standard Bellman equation into the **soft Bellman equation**. SAC uses a separate "soft" Q-function, and its objective is to learn a policy and a Q-function that satisfy this equation

B. Exploration vs. Exploitation

GP-TS achieves exploration through uncertainty-aware sampling: by drawing a sample function from the GP posterior, it can explore under-sampled regions of the hyperparameter space. In our experiments, GP-TS often tried diverse learning rates during the early training stages. Once the posterior concentrated on promising regions, the policy shifted to exploitation.

TD3, on the other hand, relies on stochasticity in action selection—Gaussian noise is added to the policy output to encourage exploration, with the noise scale decaying over time. Early training phases are more exploratory, while later stages rely increasingly on learned Q-values to guide exploitation. This mechanism worked effectively in our setting, though its efficiency depends on careful noise tuning and reward shaping.

SAC is a sophisticated dance between three distinct optimizations: training critics to estimate the reward-plus-entropy value (soft Q-value), training the actor to produce actions that maximize this soft value, and often, tuning a temperature parameter to maintain a healthy balance between exploitation (reward) and exploration (entropy).

C. Empirical Performance Comparison

In terms of convergence speed, TD3 /TRPO/SAC can generally achieved higher F1 scores in the early epochs compared to GP-TS. This is attributed to its capacity to immediately react to training dynamics through policy feedback. However, as training progressed, GP-TS gradually caught up, and both methods yielded competitive final results.

On average, GP-TS exhibited lower variance across random seeds, suggesting higher robustness. Deep RL algorithm's performance showed slightly higher variance due to stochasticity in both policy learning and exploration. Nonetheless, the best Deep-RL algorithms runs consistently matched or outperformed GP-TS, particularly on more complex datasets.

From a usability perspective, GP-TS is simpler to implement and tune. It operates with fewer internal hyperparameters and does not require training neural networks. In contrast, TD3/TRPO/SAC introduces additional training overhead for its actor and critic networks and requires tuning of multiple learning rates, exploration noise schedules, and buffer sizes.

D. Interpretability and Scalability

GP-TS offers greater interpretability. The GP surrogate model provides insight into the learning rate–performance relationship, and kernel parameters can quantify how sensitive performance is to scheduling. This can guide human analysis of training dynamics. Deep RL algorithms, being a black-box policy model, lacks this transparency. Analyzing why a particular action was chosen is more difficult.

On the other hand, Deep RL algorithms scales more naturally to higher-dimensional hyperparameter spaces. Extending GP-TS to multiple continuous parameters suffers from the curse of dimensionality, as GP inference becomes computationally expensive and kernel design becomes more complex. Deep RL algorithms via neural approximation, handles multi-dimensional actions more gracefully and could be extended to jointly tune learning rate, dropout, weight decay, etc.

E. Final Summary

In summary, GP-TS is a reliable, interpretable, and computationally efficient strategy for hyperparameter scheduling, especially when training data are limited and quick adaptation is desired. It excels in low-dimensional search spaces and provides meaningful uncertainty quantification.

TD3/TRPO/SAC, these Deep Learning RL algorithms, while more complex, delivers stronger adaptability and long-horizon planning. It demonstrates advantages in early convergence and flexibility in capturing non-trivial scheduling patterns, albeit at the cost of higher variance and implementation complexity.

Future systems may combine the two approaches—for example, using GP-TS to bootstrap an RL agent or using RL for fine-grained scheduling guided by GP-informed trends. Both methods underscore the promise of learning-based hyperparameter optimization for improving model robustness in noisy NLP tasks.

VI. Conclusion and Future Work

In this work, we explored the application of automated hyperparameter scheduling to distantly supervised named entity recognition (NER), a setting characterized by noisy and incomplete training labels. Our study compared two complementary techniques: Bayesian optimization using Gaussian Process Thompson Sampling (GP-TS) and deep reinforcement learning using Twin Delayed Deep Deterministic Policy Gradient (TD3) ,Deep Q Learning(DQN), Trusr-Region-Policy-Optimization(TRPO) and Soft-actor-critic(SAC).

Across three benchmark datasets—Webpage, Wikigold, and CoNLL-2003 with distant supervision—both approaches consistently outperformed static hyperparameter baselines, confirming the hypothesis that dynamic, data-driven scheduling of training parameters (in this case, the learning rate) can significantly enhance learning outcomes in noisy environments. GP-TS offered a sample-efficient way to explore and exploit the hyperparameter space with a global surrogate model, while TD3/TRPO/SAC learned an adaptive policy to respond to the evolving state of training, showing stronger performance on average due to its ability to consider temporal dependencies.

Our findings emphasize a broader insight: extitthe trajectory of training—how hyperparameters evolve over time—can be as important as the model architecture itself, especially when data quality is compromised. By shifting focus from static

optimization to adaptive scheduling, we enable models to better accommodate the complexity of weak supervision.

Future Work

Our work opens several directions for future research:

- **Multi-hyperparameter Scheduling:** While this study focused primarily on learning rate scheduling, many other hyperparameters could benefit from dynamic adjustment. These include dropout rates, weight decay, noise thresholds in robust loss functions, or optimization-related parameters like gradient clipping. Extending GP-TS to handle multiple dimensions may require structured kernels or dimensionality reduction, while TD3 can be directly adapted to multi-output policies.
- **Transferable Scheduling Policies:** RL-based scheduling policies may generalize across tasks or datasets if trained on representative training trajectories. An exciting direction involves learning policies on proxy datasets and transferring them to real-world applications via meta-learning. Similarly, one might explore transferring BO priors across similar datasets or domains.
- **Integration with Transformer-based NER Models:** Our current implementation used BiLSTM-CRF, but transformer-based architectures (e.g., BERT, RoBERTa) present additional challenges and opportunities. These models are more sensitive to fine-tuning hyperparameters, and dynamic scheduling could help prevent overfitting and catastrophic forgetting. However, their computational demands may require more efficient scheduling strategies, such as low-fidelity evaluations or asynchronous updates.
- **Theoretical Foundations:** Empirical results strongly suggest the benefit of hyperparameter scheduling under noisy labels, but a deeper theoretical understanding is needed. For instance, formal analyses could help explain why certain schedules help escape noise-induced local minima or how the learning dynamics interact with noise. Establishing theoretical guarantees for convergence and robustness under label noise could further legitimize scheduling approaches.
- **Hybrid Strategies:** There is potential to combine BO and RL into hybrid approaches. For example, one could initialize an RL policy using schedules learned via BO, or use RL to explore fine-grained schedules while BO captures global trends. Alternately, trajectories from RL agents could be used to construct GP surrogates for uncertainty-aware control. These hybrids may offer the best of both worlds: data-efficiency and long-term planning.
- **Robustness and Safety:** In safety-critical or online learning scenarios, it is essential to ensure that the scheduler does not introduce extreme changes that degrade performance catastrophically. Future work could explore constrained versions of BO and RL—for example, safe exploration techniques or bounded action spaces—and reward shaping or penalty terms to penalize risky behaviors.

Final Remarks

Automated hyperparameter scheduling holds significant promise for improving model robustness and performance in settings where labeled data is noisy, weak, or limited. By treating training as a controlled process rather than a static routine, we open the door to more adaptive and resilient learning. We believe this work will inspire further exploration at the intersection of AutoML, reinforcement learning, and natural language processing, and serve as a stepping stone toward more intelligent and autonomous training systems.

REFERENCES

[1] Isabelle Augenstein et al. Distantly supervised learning for emerging entity classification. In *ACL*, 2017.

[2] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. In *Journal of Machine Learning Research*, volume 13, pages 281–305, 2012.

[3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018.

[4] Yukun Hou, Jiaming Zhang, et al. Bond: Bert-assisted open-domain named entity recognition with distant supervision. In *EMNLP*, 2020.

[5] Max Jaderberg et al. Population based training of neural networks. In *arXiv preprint arXiv:1711.09846*, 2017.

[6] Thai Le et al. Episodic policy gradient training for reinforcement learning. In *NeurIPS*, 2022.

[7] Jiacheng Li, Aixin Zhang, et al. Salo: Self-adaptive label optimization for noisy named entity recognition. In *ACL Findings*, 2022.

[8] Stephen Mayhew, Chen-Tse Tsai, and Dan Roth. Named entity recognition with partially annotated training data. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[9] Nanyun Peng and Mark Dredze. Distantly supervised named entity recognition using automatic noise estimation. In *ACL*, 2019.

[10] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in neural information processing systems*, volume 25, 2012.

[11] Honglin Xu and Alan Fern. Learning rate adaptation with the reinforcement learning of learning rates. In *International Conference on Learning Representations (ICLR)*, 2019.